Problem Statement:

To apply the artificial immune pattern recognition to perform a task of structure damage Classification.

In [1]:
```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
```

Create / Load Dataset

Simulated dataset: [crack_length, vibration, stress] → damage level

In [2]:
```
np.random.seed(42)
data = {
    'crack_length': [2, 5, 8, 1, 7, 6, 3, 9, 4, 2],
    'vibration': [10, 20, 30, 8, 28, 25, 15, 35, 18, 12],
    'stress': [100, 200, 300, 90, 280, 260, 150, 350, 170, 110],
    'damage': ['Low', 'Medium', 'High', 'Low', 'High', 'High', 'Medium', 'High', 'Medium', 'Low']
}

df = pd.DataFrame(data)
```

Preprocessing

In [3]:
```
# Encode labels
df['damage'] = df['damage'].map({'Low':0, 'Medium':1, 'High':2})
```

In [4]:
```
# Features & labels
X = df.drop('damage', axis=1)
y = df['damage']
```

In [5]:
```
# Normalization
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

In [6]:
```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y)
```

AIPR Model (Core Logic)

In [7]:
```
# Initialize Antibodies

class AIPR:
    def __init__(self, num_antibodies=3):
        self.num_antibodies = num_antibodies
        self.antibodies = None
        self.labels = None

        # Train (Affinity + Selection)
    def fit(self, X, y):
        # Initialize antibodies randomly from data
        indices = np.random.choice(len(X), self.num_antibodies, replace=False)
        self.antibodies = X[indices]
        self.labels = y.iloc[indices].values

        # Affinity maturation (simple version)
        for _ in range(30):  # iterations
            for i in range(len(X)):
                antigen = X[i]
                label = y.iloc[i]

                # Compute distance (affinity)
                distances = np.linalg.norm(self.antibodies - antigen, axis=1)
                best_idx = np.argmin(distances)

                # Update antibody (move closer if correct)
                if self.labels[best_idx] == label:
                    self.antibodies[best_idx] += 0.1 * (antigen - self.antibodies[best_idx])
                else:
                    # Mutation (small random change)
```

```
                    self.antibodies[best_idx] += np.random.normal(0, 0.01, size=antigen.shape)
        # Prediction
    def predict(self, X):
        predictions = []
        for antigen in X:
            distances = np.linalg.norm(self.antibodies - antigen, axis=1)
            best_idx = np.argmin(distances)
            predictions.append(self.labels[best_idx])
        return np.array(predictions)
```

Train & Test

In [8]:
```
model = AIPR(num_antibodies=6)
model.fit(X_train, y_train)
```

In [9]:
```
y_pred = model.predict(X_test)
```

In [10]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 1.0
```

In [11]:
```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         1
           2       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```